

EDU Backend API Documentation

Django REST Framework · JWT Auth · SQLite

Version 1.0 · May 2026

This document describes the complete REST API for the EDU platform backend. It covers authentication, file management, and AI-powered chat endpoints. All protected endpoints require a valid JWT access token in the Authorization header.

1. Project Overview

Technology Stack

Component	Technology	Details
Language	Python 3.14	Backend language
Framework	Django 5.2	Web framework
API Layer	Django REST Framework 3.17	REST API toolkit
Authentication	SimpleJWT	JWT token auth
Database	SQLite	db.sqlite3 file
File Storage	Local filesystem	media/ folder
CORS	django-cors-headers	Frontend access

Base URL

http://127.0.0.1:8000/api/

All API endpoints are prefixed with /api/. In production, replace 127.0.0.1:8000 with your server domain.

Project Folder Structure

edu_backend/ api/ models.py – Database models serializers.py – JSON conversion views.py – API logic urls.py – API routes mysite/ settings.py – Configuration urls.py – Root URL config media/ – Uploaded files db.sqlite3 – Database manage.py – Django CLI tool

2. Authentication

The API uses JWT (JSON Web Tokens) for authentication. After logging in, the client receives an access token and a refresh token. The access token must be included in the Authorization header of every protected request.

How to send the token

```
Authorization: Bearer <your_access_token>
```

Token lifetimes

Token	Lifetime	Purpose
access	5 minutes	Sent with every API request
refresh	1 day	Used to get a new access token

Endpoints

POST /api/auth/register/

Auth: No

Create a new user account. Returns JWT tokens immediately after registration.

Request body (JSON):

```
{ "username": "john", "email": "john@example.com", "password": "securepassword" }
```

Response (200 / 201):

```
{ "refresh": "eyJhbGciOi...", "access": "eyJhbGciOi..." }
```

POST /api/auth/login/

Auth: No

Login with existing credentials. Returns JWT access and refresh tokens.

Request body (JSON):

```
{ "username": "john", "password": "securepassword" }
```

Response (200 / 201):

```
{ "refresh": "eyJhbGciOi...", "access": "eyJhbGciOi..." }
```

POST**/api/auth/refresh/****Auth: No**

Get a new access token using a valid refresh token. Call this when the access token expires.

Request body (JSON):

```
{ "refresh": "eyJhbGciOi..." }
```

Response (200 / 201):

```
{ "access": "eyJhbGciOi..." }
```

3. File Management

Users can upload files and view their upload history. Files are stored on the server under the media/ folder. Only PDF, DOCX, and PPTX formats are accepted.

Allowed file types

Extension	MIME Type	Description
.pdf	application/pdf	PDF documents
.docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document	Word documents
.pptx	application/vnd.openxmlformats-officedocument.presentationml.presentation	PowerPoint files

Endpoints

POST /api/files/

Auth: Yes

Upload a file. Must be sent as multipart/form-data (not JSON). Only authenticated users can upload. Files are tied to the logged-in user.

■ *Send as form-data, NOT JSON. The field name must be exactly 'file'.*

Request body (JSON):

```
Content-Type: multipart/form-data field name: file value:
```

Response (200 / 201):

```
{ "id": 1, "original_name": "report.pdf", "file_type": "pdf", "uploaded_at":  
  "2026-05-03T18:00:00Z", "file": "/media/uploads/report.pdf" }
```

GET /api/files/

Auth: Yes

Get the full file upload history for the currently logged-in user. Returns files ordered by most recently uploaded first.

Response (200 / 201):

```
[ { "id": 1, "original_name": "report.pdf", "file_type": "pdf", "uploaded_at":  
  "2026-05-03T18:00:00Z", "file": "/media/uploads/report.pdf" } ]
```

4. Chat

Each user can have multiple chat sessions. Every session contains a list of messages with roles of either 'user' or 'ai'. Messages can optionally reference an uploaded file so the AI can process it.

Chat flow

Step	Action	Endpoint
1	Create a new chat session	POST /api/chat/sessions/
2	Send a message (with optional file)	POST /api/chat/sessions/{id}/message
3	Read chat history	GET /api/chat/sessions/{id}/messages/
4	List all sessions	GET /api/chat/sessions/

Endpoints

GET /api/chat/sessions/

Auth: Yes

Returns all chat sessions belonging to the logged-in user, ordered by most recent first.

Response (200 / 201):

```
[ { "id": 1, "title": "Summarize my report", "created_at": "2026-05-03T18:00:00Z",  
  "messages": [] } ]
```

POST /api/chat/sessions/

Auth: Yes

Create a new chat session. The title is optional — defaults to 'New Chat'.

Request body (JSON):

```
{ "title": "Summarize my report" // optional }
```

Response (200 / 201):

```
{ "id": 1, "title": "Summarize my report", "created_at": "2026-05-03T18:00:00Z",  
  "messages": [] }
```

GET /api/chat/sessions/{id}/messages/

Auth: Yes

Retrieve the full message history for a specific chat session. The {id} is the session ID returned when creating a session.

Response (200 / 201):

```
[ { "id": 1, "role": "user", "content": "Summarize this file", "file": 1, "created_at":  
  "..."}, { "id": 2, "role": "ai", "content": "The file is about...", "file": null,  
  "created_at": "..."} ]
```

POST**/api/chat/sessions/{id}/messages/****Auth: Yes**

Send a user message to a session. The AI team's logic runs here and an AI reply is returned in the same response. Optionally attach a previously uploaded file by its ID.

■ *The AI response is currently a placeholder. The AI team will replace it with real logic in views.py.*

Request body (JSON):

```
{ "content": "Please summarize this document", "file_id": 1 // optional – ID from file upload }
```

Response (200 / 201):

```
{ "user_message": { "id": 3, "role": "user", "content": "Please summarize...", "created_at": "..."}, "ai_message": { "id": 4, "role": "ai", "content": "The document covers...", "created_at": "..."} }
```

5. Database Models

User (built-in Django model)

Field	Type	Description
id	Integer (auto)	Primary key
username	String	Unique username
email	String	Email address
password	String (hashed)	Stored as bcrypt hash — never plain

UploadedFile

Field	Type	Description
id	Integer (auto)	Primary key
user	FK → User	Owner of the file
file	FileField	Path to file in media/uploads/
original_name	String	Original filename from the user
file_type	String	pdf, docx, or pptx
uploaded_at	DateTime	Auto-set on upload

ChatSession

Field	Type	Description
id	Integer (auto)	Primary key
user	FK → User	Session owner
title	String	Display name for the chat
created_at	DateTime	Auto-set on creation

ChatMessage

Field	Type	Description
id	Integer (auto)	Primary key
session	FK → ChatSession	Parent session
role	String (user / ai)	Who sent this message
content	Text	Message body
file	FK → UploadedFile	Optional attached file
created_at	DateTime	Auto-set on creation

6. Quick Reference — All Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register/	No	Create account
POST	/api/auth/login/	No	Login, get JWT
POST	/api/auth/refresh/	No	Refresh access token
POST	/api/files/	Yes	Upload PDF/DOCX/PPTX
GET	/api/files/	Yes	Get file history
GET	/api/chat/sessions/	Yes	List all chat sessions
POST	/api/chat/sessions/	Yes	Create new session
GET	/api/chat/sessions/{id}/messages/	Yes	Get chat history
POST	/api/chat/sessions/{id}/messages/	Yes	Send message to AI

Error responses

HTTP Code	Meaning	Common cause
200 OK	Success	GET request worked
201 Created	Created	POST created a resource
400 Bad Request	Validation error	Missing or invalid fields
401 Unauthorized	Not authenticated	Missing or expired JWT token
403 Forbidden	No permission	Trying to access another user's data
404 Not Found	Resource missing	Wrong ID or URL
405 Method Not Allowed	Wrong HTTP method	Using GET when POST is required

Notes for the frontend developer

- 1 Always send the Authorization header as: Bearer <access_token>
- 2 File uploads must use multipart/form-data, not application/json
- 3 When the access token expires (401), call /api/auth/refresh/ to get a new one
- 4 The file field in responses is a relative URL — prepend the base URL to get the full path
- 5 CORS is enabled — the frontend can call this backend from any origin during development

Notes for the AI team

Where to plug in

Open `api/views.py` and find the `ChatMessageView` class.
In the `post()` method, replace the placeholder `ai_msg bloc`
You have access to: `request.user`, the session, the user's